SAPTHAGIRI NPS UNIVERSITY

Industry Oriented Technical Training for B.E (2028 Batch) — Phase I

DAY 1

Concept Reference — Combined

Part A: Java Foundations & the Problem-Solving Mindset (BASIC)

Part B: DSA BASIC — Inheritance, Abstraction, Constructor & Interface

Concept · Real-Life Example · Syntax · Mini Problems · LeetCode Practice

A Agenda — Part A: Java Foundations BASIC

1. Abstraction
2. Encapsulation
3. Inheritance
4. Polymorphism
5. Control Flow & Systematic Problem Decomposition
6. Methods, Recursion & Thinking in Stacks
7. Fast I/O Template
8. Armstrong Number
9. Palindrome Number
10. GCD by Euclid's Algorithm
11. Pattern Printing

B Agenda — Part B: DSA BASIC

12. Types of Inheritance
13. Using super for Constructor and Method Access
14. this() and super() — Constructor Chaining
15. Interface-Based Abstraction
16. Chaining OOP for Clean, Testable Solutions
17. Collections
18. Generics
19. Streams
20. Functional Java — Lambdas & Functional Interfaces
21. Interface vs Abstract Class
22. Recursive Tower of Hanoi (+ Recursion Tree)
23. Recursive Sum of Digits (+ Recursion Tree)
24. Recursive String Reversal (+ Recursion Tree)
25. Constructor Overloading
26. Compile-Time vs Runtime Polymorphism (Practice)
27. Method Overriding

PART A

Java Foundations & the Problem-Solving Mindset (BASIC)

Part A · Sections 1 – 11

1 Abstraction

~15 min

CONCEPT

- Abstraction means exposing only the relevant behaviour of an object to the outside world while hiding the internal implementation details.
- In Java it is achieved using abstract classes (abstract keyword, may mix implemented + unimplemented methods) and interfaces (a pure contract of "what" a type can do).

1 Abstraction

~15 min

REAL-LIFE EXAMPLE

ANALOGY

Driving a car: you use the steering wheel, accelerator, and brake without knowing how fuel injection, the ECU, or the transmission internally work. The dashboard is the abstraction; the engine bay is the hidden implementation.

1 Abstraction

~15 min

JAVA SYNTAX

```
abstract class Vehicle {
    abstract void start();          // WHAT it does, not HOW
    void honk() { System.out.println("Beep!"); }
}

class Car extends Vehicle {
    void start() { System.out.println("Turning ignition key..."); }
}

interface Payable {
    void pay(double amount);        // 100% contract, no state
}
```

1 Abstraction

~15 min

MINI PROBLEMS

1. Design an abstract class Shape with an abstract method area(); implement it in Circle and Rectangle classes and print both areas from a Shape reference.

2. Design an interface Payable with method pay(double amount); implement it in a CashPayment class and a CardPayment class, each printing a different confirmation message.

1 Abstraction

~15 min

LEETCODE PRACTICE

Design Parking System Easy

1603

Forces you to design a class around a clean public contract (addCar) while hiding the slot-counting logic inside.

Design HashSet Easy

705

You expose only add/remove/contains — the underlying bucket storage is fully hidden, the essence of abstraction.

Trainer Note: *LeetCode doesn't tag problems "abstraction" directly — the closest fit is the "Design ___" family, where you must expose a minimal public API and hide the internal storage/logic.*

2 Encapsulation

~15 min

CONCEPT

- Encapsulation means bundling data (fields) with the methods that operate on that data, and restricting direct access to the fields using access modifiers (typically private) plus controlled public getters/setters.
- The goal is to protect an object's internal state from invalid values by forcing all changes to go through validated methods.

2 Encapsulation

~15 min

REAL-LIFE EXAMPLE

ANALOGY

A medicine capsule: the active ingredients are sealed inside a shell. You swallow the capsule as a whole — you never handle the loose powder directly. The shell is the access control.

2 Encapsulation

~15 min

JAVA SYNTAX

```
class BankAccount {
    private double balance;                // hidden state

    public void deposit(double amt) {
        if (amt > 0) balance += amt;        // validation
    }
    public boolean withdraw(double amt) {
        if (amt > 0 && amt <= balance) { balance -= amt; return true; }
        return false;
    }
    public double getBalance() { return balance; }  // controlled read access
}
```

2 Encapsulation

~15 min

MINI PROBLEMS

1. Build a Student class with a private int[] marks; add addMark(int m) that rejects marks outside 0-100, and getAverage() that returns the average.

2. You are given a Rectangle class with public fields length and width. Refactor it into a properly encapsulated class with private fields and setters that reject non-positive values.

2 Encapsulation

~15 min

LEETCODE PRACTICE

Min Stack Medium

155

Internal state (the stack, the running minimum) is fully private; only push/pop/top/getMin are exposed.

Implement Queue using Stacks Easy

232

The two internal stacks are private implementation detail; the class exposes only push/pop/peek/empty.

3 Inheritance

~20 min

CONCEPT

- Inheritance lets a class (subclass/child) reuse and extend the fields and methods of another class (superclass/parent) using the extends keyword.
- Java supports single, multilevel, and hierarchical inheritance for classes, but NOT multiple inheritance of classes (a class cannot extend two classes) — this avoids the diamond problem. A class CAN implement multiple interfaces instead.

3 Inheritance

~20 min

REAL-LIFE EXAMPLE

ANALOGY

A family tree of professions: a Doctor 'is-a' Person (inherits name, age); a Surgeon 'is-a' Doctor (inherits everything a Doctor has, plus adds operateOn()). Each generation reuses what came before and adds something new.

3 Inheritance

~20 min

JAVA SYNTAX

```
class Employee {
    String name;
    double baseSalary;
    double calculateSalary() { return baseSalary; }
}

class Manager extends Employee {          // single inheritance
    double bonus;
    @Override
    double calculateSalary() { return baseSalary + bonus; }
}

class SeniorManager extends Manager { }   // multilevel inheritance
```

3 Inheritance

~20 min

MINI PROBLEMS

1. Model Person -> Student -> GraduateStudent (multilevel). Each level should add exactly one new field and one new method that uses it.

2. Model a hierarchical Shape -> Circle and Shape -> Square, where Shape has a common describe() method that both children inherit unchanged.

3 Inheritance

~20 min

LEETCODE PRACTICE

Peeking Iterator Medium

284

Your class implements/extends the Iterator contract and adds a peek() behaviour on top — interface-based extension in Java's style.

Trainer Note: *LeetCode is intentionally weak on testing class-to-class inheritance chains — its problems are single-class by design. The mini problems above (done on paper/IDE, not submitted to a judge) are the real assessment for this topic; treat the LeetCode row as a loose warm-up only.*

4 Polymorphism

~20 min

CONCEPT

- Polymorphism means "many forms" — the same method name behaving differently depending on context.
- Compile-time polymorphism (method overloading): same method name, different parameter lists, resolved by the compiler.
- Runtime polymorphism (method overriding): a subclass redefines a parent method with the identical signature; the JVM decides which version to run based on the actual object type at runtime (dynamic dispatch).

4 Polymorphism

~20 min

REAL-LIFE EXAMPLE

ANALOGY

A universal remote's 'power' button: pressing it does something different depending on which device is currently selected (TV, AC, set-top box) — same action name, different behaviour per device, decided at the moment you press it.

4 Polymorphism

~20 min

JAVA SYNTAX

```
class Calculator {
    int area(int side) { return side * side; }              // overload 1
    double area(double l, double b) { return l * b; }       // overload 2 (compile-time)
}

class Animal { void sound() { System.out.println("..."); } }
class Dog extends Animal {
    @Override void sound() { System.out.println("Bark"); }   // overriding (runtime)
}
```

4 Polymorphism

~20 min

MINI PROBLEMS

1. Overload a print() method to accept an int, a String, and a double, each formatting the output slightly differently.

2. Override toString() in a custom Point class and compare the printed output to the default Object.toString() before your override.

4 Polymorphism

~20 min

LEETCODE PRACTICE

Implement Queue using Stacks Easy

232

Same queue interface (push/pop/peek), different underlying implementation than a normal array-backed queue — a taste of runtime polymorphism via interfaces.

Design a Stack With Increment Operation Medium

1381

You extend a familiar Stack contract (push/pop) with a new increment() behaviour, similar to how overriding extends/changes inherited behaviour.

Trainer Note: *As with inheritance, LeetCode doesn't directly grade overloading/overriding — use the mini problems above for that; the LeetCode rows here practise the surrounding "same interface, different behaviour" mindset.*

5 Control Flow & Systematic Problem Decomposition

~25 min

CONCEPT

- Control flow (if/else, switch, loops) is the mechanism; decomposition is the discipline of using that mechanism correctly by planning before coding.
- The decompose-then-code method: (1) read the problem twice, (2) identify inputs/outputs, (3) break into sub-steps in plain English/pseudocode, (4) list edge cases, (5) translate to Java step by step, (6) test against samples and edge cases.

5 Control Flow & Systematic Problem Decomposition

~25 min

REAL-LIFE EXAMPLE

ANALOGY

Following a recipe: a good cook reads the whole recipe first, lists out ingredients (inputs) and the final dish (output), then works through it step by step — they don't start chopping vegetables before knowing what the final dish even is.

5 Control Flow & Systematic Problem Decomposition

~25 min

JAVA SYNTAX

```
// Decomposition written as comments BEFORE code — required habit from Day 1 onward
// Step 1: read n
// Step 2: loop i from 1 to n, track largest and secondLargest
// Step 3: edge case -> n < 2 means no second largest exists
int n = arr.length;
if (n < 2) { System.out.println("No second largest"); return; }
```

5 Control Flow & Systematic Problem Decomposition

~25 min

MINI PROBLEMS

1. Decompose (write the 6 steps as comments) "find the second largest number in an array" before writing any code.

2. Decompose "check whether three given side lengths can form a valid triangle" before writing any code.

5 Control Flow & Systematic Problem Decomposition

~25 min

LEETCODE PRACTICE

Two Sum Easy

1

Classic first decomposition exercise: identify input/output clearly before jumping to the brute-force vs hashmap approach.

Missing Number Easy

268

Forces edge-case thinking (missing number could be 0 or n) — exactly the step-4 habit being taught.

6 Methods, Recursion & Thinking in Stacks

~30 min

CONCEPT

- A recursive method calls itself, built from a base case (stops the recursion) and a recursive case (reduces the problem and calls itself again).
- Every call pushes a new frame onto the call stack; the base case is what lets the stack start popping (returning) instead of only growing. A missing/unreachable base case causes a StackOverflowError.
- Mental model: trust that the smaller subproblem already has the right answer — your only job is to describe how the current step relates to that smaller answer.

6 Methods, Recursion & Thinking in Stacks

~30 min

REAL-LIFE EXAMPLE

ANALOGY

Russian nesting dolls (matryoshka): to open the whole set you open the outer doll, which reveals a slightly smaller doll inside, and you repeat the exact same 'open it' action — until you reach the smallest doll that doesn't open any further (the base case).

6 Methods, Recursion & Thinking in Stacks

~30 min

JAVA SYNTAX

```
int factorial(int n) {
    if (n == 0) return 1;          // base case
    return n * factorial(n - 1);   // recursive case
}

// Call stack for factorial(4):
// factorial(4) -> factorial(3) -> factorial(2) -> factorial(1) -> factorial(0) returns 1
// then each frame returns: 1*1=1, 2*1=2, 3*2=6, 4*6=24
```

6 Methods, Recursion & Thinking in Stacks

~30 min

MINI PROBLEMS

1. Hand-trace what power(2, 5) returns given power(b, e) = b * power(b, e-1), base case e == 0 -> 1. Draw the call stack.

2. Write both an iterative and a recursive version of factorial(n) and compare which is easier to read and why.

6 Methods, Recursion & Thinking in Stacks

~30 min

LEETCODE PRACTICE

509 Fibonacci Number Easy

Simplest two-branch recursion after factorial — good next step for call-stack reasoning.

70 Climbing Stairs Easy

Same recursive shape as Fibonacci in a word-problem wrapper — reinforces translating a story into a recurrence.

344 Reverse String Easy

Solve it recursively (swap ends, recurse inward) to practise base-case + shrinking-problem thinking on an array instead of a number.

Trainer Note: *Keep in-class recursion examples to one self-call (factorial, sum of digits). Two-call recursion (Fibonacci-style) and full recursion-tree drawing are Day 2 hands-on content — today only builds the single-call mental model.*

7 Fast I/O Template

~20 min

CONCEPT

- Scanner performs regex-based token parsing on every read, which becomes a bottleneck once N grows large. Competitive judges (Codeforces, HackerEarth, GFG) commonly reject Scanner-based solutions on tight time limits.
- BufferedReader reads raw text; combined with StringTokenizer for splitting, it avoids the regex overhead entirely.

7 Fast I/O Template

~20 min

REAL-LIFE EXAMPLE

ANALOGY

Scanner is like reading a book one word at a time, checking a dictionary after every single word. BufferedReader is like reading a full page silently first, then splitting it into words yourself — much faster when there's a lot to read.

7 Fast I/O Template

~20 min

JAVA SYNTAX

```
import java.io.*;
import java.util.*;

BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
int n = Integer.parseInt(br.readLine().trim());
StringTokenizer st = new StringTokenizer(br.readLine());
long sum = 0;
int min = Integer.MAX_VALUE, max = Integer.MIN_VALUE;
for (int i = 0; i < n; i++) {
    int x = Integer.parseInt(st.nextToken());
    sum += x; min = Math.min(min, x); max = Math.max(max, x);
}
```

7 Fast I/O Template

~20 min

MINI PROBLEMS

1. Extend the template to also print the average of the N integers as a double, correctly rounded to 2 decimal places.

2. Modify the template to handle T test cases: read T first, then repeat the N-integers-sum/min/max routine T times.

Trainer Note: *LeetCode handles I/O for you automatically, so there's no direct LeetCode equivalent for this skill — it matters specifically for judges like Codeforces/HackerEarth/GFG that this course's competitive-coding phase (Day 13 onward) will use.*

8 Armstrong Number

~15 min

CONCEPT

- A number is an Armstrong number if it equals the sum of its own digits, each raised to the power of the total digit count (e.g. 153 = $1^3 + 5^3 + 3^3$).
- Decomposition: count digits -> extract each digit -> raise to power = digit count -> accumulate -> compare to the original.

8 Armstrong Number

~15 min

REAL-LIFE EXAMPLE

ANALOGY

Think of it as a signature check: you take a number apart digit by digit, apply the exact same transformation to each piece, put it back together, and see if you get the original number back — like verifying a fingerprint reconstructs to the same pattern.

8 Armstrong Number

~15 min

JAVA SYNTAX

```
int n = 153, original = n, digits = String.valueOf(n).length();
int sum = 0;
while (n > 0) {
    int d = n % 10;
    sum += (int) Math.pow(d, digits);
    n /= 10;
}
System.out.println(sum == original ? "Armstrong" : "Not Armstrong");
```

8 Armstrong Number

~15 min

MINI PROBLEMS

1. Print all Armstrong numbers between 1 and 1000.

2. Extend the check to 6-digit numbers and explain why using int math with Math.pow() needs care (precision/casting) at that size.

8 Armstrong Number

~15 min

LEETCODE PRACTICE

Add Digits Easy

258

Same digit-extraction loop skill (n % 10, n / 10) applied to a different digit-sum rule.

Sum of Digits in Base K Easy

1837

Generalises today's base-10 digit extraction to an arbitrary base — a natural stretch problem.

9 Palindrome Number

~15 min

CONCEPT

- A number (or string) is a palindrome if it reads the same forwards and backwards.
- For numbers: reverse the digits arithmetically (no string conversion needed) and compare the reversed value to the original.

9 Palindrome Number

~15 min

REAL-LIFE EXAMPLE

ANALOGY

Like the word 'MADAM' or a mirror held up to the number — read it left-to-right and right-to-left; if both readings match exactly, it's a palindrome.

9 Palindrome Number

~15 min

JAVA SYNTAX

```
int n = 121, original = n, reversed = 0;
while (n > 0) {
    reversed = reversed * 10 + n % 10;
    n /= 10;
}
System.out.println(reversed == original ? "Palindrome" : "Not Palindrome");
```

9 Palindrome Number

~15 min

MINI PROBLEMS

1. Adapt the same idea to check whether a given String is a palindrome, ignoring case.

2. Count how many palindrome numbers exist in a given range [L, R].

9 Palindrome Number

~15 min

LEETCODE PRACTICE

Palindrome Number Easy

9

Direct match — today's exact in-class solution, with the added constraint of solving it without converting to a String.

Valid Palindrome Easy

125

Same palindrome-check idea applied to a string with punctuation/case to strip first — natural next step.

10 GCD by Euclid's Algorithm

~15 min

CONCEPT

- The Greatest Common Divisor (GCD) of two numbers is the largest number that divides both exactly.
- Euclid's algorithm: gcd(a, b) = gcd(b, a % b), with base case gcd(a, 0) = a. Each step shrinks the problem, just like the recursion taught earlier in the day.

10 GCD by Euclid's Algorithm

~15 min

REAL-LIFE EXAMPLE

ANALOGY

Tiling a rectangular floor with the largest possible square tiles with no leftover space and no cutting: the side length of the largest such square is exactly the GCD of the floor's two dimensions.

10 GCD by Euclid's Algorithm

~15 min

JAVA SYNTAX

```
static int gcd(int a, int b) {
    if (b == 0) return a;    // base case
    return gcd(b, a % b);
}
```

10 GCD by Euclid's Algorithm

~15 min

MINI PROBLEMS

1. Extend gcd() to compute the LCM using the relation lcm(a, b) = (a * b) / gcd(a, b).

2. Find the GCD of three numbers by applying gcd() pairwise: gcd(gcd(a, b), c).

10 GCD by Euclid's Algorithm

~15 min

LEETCODE PRACTICE

Greatest Common Divisor of Strings Easy

1071

Applies the exact same Euclidean idea to string lengths instead of integers — a great "same algorithm, new domain" exercise.

Insert Greatest Common Divisors in Linked List Medium

2807

Direct reuse of today's gcd() function inside a slightly bigger linked-list problem — good stretch/homework problem.

11 Pattern Printing

~15 min

CONCEPT

- Pattern problems train nested-loop control: the outer loop controls rows, the inner loop controls what gets printed in each row, based on a rule tied to the row number.
- Always decompose first: how many rows? what changes per row? what's printed in the inner loop and how many times?

11 Pattern Printing

~15 min

REAL-LIFE EXAMPLE

ANALOGY

Laying bricks in a stepped pyramid wall: each row you build has a clear, predictable number of bricks based on which row you're on — you follow a simple per-row rule rather than placing every brick individually from scratch.

11 Pattern Printing

~15 min

JAVA SYNTAX

```
// Right-angled triangle of stars, n rows
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) System.out.print("*");
    System.out.println();
}
```

11 Pattern Printing

~15 min

MINI PROBLEMS

1. Print a hollow square pattern of stars of side n (border only, blank inside).

2. Print a number pattern where row r prints the digit r exactly r times (e.g. row 3 -> "3 3 3").

11 Pattern Printing

~15 min

LEETCODE PRACTICE

Pascal's Triangle Easy

118

A numeric row-by-row pattern where each row's values depend on the previous row — a natural step up from static star/number patterns.

PART B

DSA BASIC:

Inheritance, Abstraction, Constructor & Interface

Part B · Sections 12 – 27

12 Types of Inheritance

~15 min

CONCEPT

- Java classes support single inheritance (one child, one parent), multilevel inheritance (a chain: grandparent -> parent -> child), and hierarchical inheritance (one parent, many independent children).
- Java does NOT allow a class to extend two classes at once (no multiple inheritance of classes) — this avoids the diamond problem. The equivalent is achieved safely through interfaces (§15), where a class can implement several of them.

12 Types of Inheritance

~15 min

REAL-LIFE EXAMPLE

ANALOGY

An organisation chart: single inheritance is a Manager reporting to one Director; multilevel is Associate -> Senior Associate -> Team Lead, each level adding responsibility; hierarchical is one Director with several independent Managers reporting to them, each specializing differently.

12 Types of Inheritance

~15 min

JAVA SYNTAX

```
// Single
class Vehicle { void move() { System.out.println("moving"); } }
class Car extends Vehicle { }

// Multilevel
class SportsCar extends Car { void turbo() { System.out.println("boost!"); } }

// Hierarchical
class Truck extends Vehicle { }
class Bike extends Vehicle { }
```

12 Types of Inheritance

~15 min

MINI PROBLEMS

1. Build a multilevel hierarchy Vehicle -> Car -> SportsCar, where each level adds exactly one new field and one new method.

2. Build a hierarchical structure with Shape as the parent and Circle, Square as two independent children, each overriding a common describe() method differently.

12 Types of Inheritance

~15 min

LEETCODE PRACTICE

Design Linked List Medium

707

Not literal class inheritance, but forces you to design a consistent Node/list structure that could naturally be extended (e.g. a doubly-linked variant) the way a class hierarchy would.

Trainer Note: *LeetCode's single-class judge format means it can't directly grade a multi-class inheritance chain — the two mini problems above are the real assessment for this topic; treat the LeetCode row as a loosely related structural warm-up only.*

13 Using super for Constructor and Method Access

~15 min

CONCEPT

- super() (as the first statement in a subclass constructor) calls the immediate parent class's constructor, so the parent's fields are initialised before the child adds its own.
- super.methodName() calls the parent's version of a method, even if the child has overridden it — useful when the child wants to extend rather than fully replace the parent's behaviour.

13 Using super for Constructor and Method Access

~15 min

REAL-LIFE EXAMPLE

ANALOGY

Renovating a house: you don't rebuild the foundation and plumbing from scratch (super() reuses the parent's setup) — you build new rooms on top of what's already there. If you want the original paint job plus a new mural, you keep the base coat (super.method()) and add your own layer on top.

13 Using super for Constructor and Method Access

~15 min

JAVA SYNTAX

```
class Person {
    String name;
    Person(String name) { this.name = name; }
    void describe() { System.out.println("Person: " + name); }
}

class Employee extends Person {
    double salary;
    Employee(String name, double salary) {
        super(name);               // parent constructor
        this.salary = salary;
    }
    @Override
    void describe() {
        super.describe();          // reuse parent's version first
        System.out.println("Salary: " + salary);
    }
}
```

13 Using super for Constructor and Method Access

~15 min

MINI PROBLEMS

1. Create a Person class and a Student class that extends it; Student's constructor must call super(name) and add a rollNo field.

2. In an overridden method, call super.method() first (to log/print the parent's behaviour) before adding the child's own extra behaviour — verify both outputs appear in the right order.

Trainer Note: *super() / super.method() mechanics are a single-class-judge blind spot for LeetCode — there's no meaningful problem number to assign here. Use the two mini problems above as the graded exercise for this topic.*

14 this() and super() — Constructor Chaining

~15 min

CONCEPT

- this() calls another constructor in the SAME class — used to avoid duplicating setup code across overloaded constructors.
- super() calls the immediate parent class's constructor. Both this() and super() must be the very first statement in a constructor, and a constructor can use at most one of the two (never both).

14 this() and super() — Constructor Chaining

~15 min

REAL-LIFE EXAMPLE

ANALOGY

Ordering a combo meal: a 'large combo' order internally reuses the 'medium combo' setup and just adds one more item, rather than re-specifying every item from scratch each time — that reuse-by-calling-a-simpler-version-of-yourself is exactly what this() does.

14 this() and super() — Constructor Chaining

~15 min

JAVA SYNTAX

```
class Rectangle {
    double length, width;
    Rectangle() {
        this(1, 1);                 // chains to the 2-arg constructor
    }
    Rectangle(double side) {
        this(side, side);           // chains to the 2-arg constructor
    }
    Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }
}
```

14 this() and super() — Constructor Chaining

~15 min

MINI PROBLEMS

1. Write a Rectangle class with 3 constructors (no-arg -> unit square, one-arg -> square of given side, two-arg -> full rectangle) chained together using this().

2. Write a Student class whose full constructor (name, rollNo, marks) calls a partial constructor (name, rollNo) via this(), defaulting marks to 0.

Trainer Note: *As with super(), constructor chaining isn't something a single-function LeetCode judge can assess — the mini problems above are the real practice for this topic.*

15 Interface-Based Abstraction

~20 min

CONCEPT

- An interface defines a pure contract — method signatures with no implementation (plus, since Java 8, optional default/static methods with bodies) and no instance state.
- A class can implement multiple interfaces, which is how Java achieves the effect of multiple inheritance safely, without the diamond-problem ambiguity that multiple class inheritance would cause.

15 Interface-Based Abstraction

~20 min

REAL-LIFE EXAMPLE

ANALOGY

The USB standard: a mouse, a keyboard, and a flash drive are completely different devices internally, but they all honour the same USB interface contract — so any of them plugs into any USB port. The port doesn't care about the internal implementation, only that the contract is honoured.

15 Interface-Based Abstraction

~20 min

JAVA SYNTAX

```
interface Drivable { void drive(); }
interface Flyable   { void fly(); }

class FlyingCar implements Drivable, Flyable {
    public void drive() { System.out.println("Driving on road"); }
    public void fly()   { System.out.println("Flying in air"); }
}
```

15 Interface-Based Abstraction

~20 min

MINI PROBLEMS

1. Define interfaces Payable and Refundable, then implement both in a single Order class.

2. Define an interface Shape2D with area() and perimeter(); implement it in two unrelated classes (e.g. Circle and Triangle) and call both through the interface type.

15 Interface-Based Abstraction

~20 min

LEETCODE PRACTICE

Design an Ordered Stream Easy

1656

You expose a single clean insert() contract while the internal pointer/array bookkeeping stays completely hidden — abstraction via a minimal interface.

Design Circular Queue Medium

622

A well-defined operation contract (enQueue/deQueue/Front/Rear) that could be implemented multiple different ways behind the same interface.

16 Chaining OOP for Clean, Testable Solutions

~20 min

CONCEPT

- This is a design habit, not a single language feature: combine abstraction, encapsulation, inheritance and polymorphism deliberately so that classes are loosely coupled and easy to test in isolation.
- Core idea — "program to an interface, not an implementation": a class should depend on an interface/abstract type rather than a concrete class, so the concrete implementation can be swapped (e.g. for a test double) without changing the dependent class.

16 Chaining OOP for Clean, Testable Solutions

~20 min

REAL-LIFE EXAMPLE

ANALOGY

Modular furniture: each module (drawer, shelf, door) is built to a standard connector interface, so you can design, build, and test any one module completely on its own workbench — you don't need the entire cabinet assembled just to check that one drawer slides smoothly.

16 Chaining OOP for Clean, Testable Solutions

~20 min

JAVA SYNTAX

```
interface Notifier { void send(String message); }

class EmailNotifier implements Notifier {
    public void send(String m) { System.out.println("Email: " + m); }
}

class OrderService {
    private final Notifier notifier;        // depends on the INTERFACE
    OrderService(Notifier notifier) { this.notifier = notifier; }
    void placeOrder() { notifier.send("Order placed!"); }
}
```

16 Chaining OOP for Clean, Testable Solutions

~20 min

MINI PROBLEMS

1. Refactor a class that directly creates (new EmailNotifier()) inside its own body so that it instead receives a Notifier through its constructor (dependency inversion).

2. Write a FakeNotifier implementing the same Notifier interface that just records messages in a list instead of sending them, and use it to verify OrderService's logic without any real email being sent.

Trainer Note: *This topic is a design discipline rather than a syntax feature, so there is no LeetCode problem set attached — the two mini problems are the intended assessment. It ties directly back to §15 (interfaces) and previews why interface-based design matters for real-world, testable Java code.*

17 Collections

~15 min

CONCEPT

- The Java Collections Framework provides ready-made data structures behind common interfaces: List (ordered, duplicates allowed — ArrayList, LinkedList), Set (no duplicates — HashSet, TreeSet), and Map (key-value pairs — HashMap, TreeMap).
- Choosing the right collection is itself a design decision: List when order/index matters, Set when uniqueness matters, Map when you need fast lookup by key.

17 Collections

~15 min

REAL-LIFE EXAMPLE

ANALOGY

A filing cabinet (List — ordered folders you can pull out by position), a stamp collection where no duplicate stamp is ever kept twice (Set), and a phonebook mapping each name to exactly one number (Map).

17 Collections

~15 min

JAVA SYNTAX

```
List<Integer> list = new ArrayList<>();
list.add(10); list.add(20);

Set<String> seen = new HashSet<>();
seen.add("apple"); seen.add("apple");   // second add is ignored

Map<String, Integer> wordCount = new HashMap<>();
wordCount.put("the", 1);
wordCount.merge("the", 1, Integer::sum); // now 2
```

17 Collections

~15 min

MINI PROBLEMS

1. Use a HashSet to find and print all duplicate elements in an int array.

2. Use a HashMap to count how many times each word appears in a given sentence.

17 Collections

~15 min

LEETCODE PRACTICE

Contains Duplicate Easy

217

Direct HashSet application — exactly today's first mini problem, phrased as a judge-graded task.

Top K Frequent Elements Medium

347

Combines a HashMap frequency count with a priority structure — a natural stretch after the word-count mini problem.

18 Generics

~15 min

CONCEPT

- Generics let a class or method be written once with a type parameter (e.g. <T>) and reused safely for any type, catching type mismatches at compile time instead of causing a ClassCastException at runtime.
- Common uses: generic container classes (Box<T>, Pair<K,V>) and generic methods that work across any Comparable type.

18 Generics

~15 min

REAL-LIFE EXAMPLE

ANALOGY

A shipping box labelled 'electronics only': the box design itself doesn't care whether it holds a phone or a laptop, but it stops you from accidentally packing a pair of socks in with the electronics — the label (type parameter) enforces consistency without the box needing a different design for every item type.

18 Generics

~15 min

JAVA SYNTAX

```
class Box<T> {
    private T value;
    void set(T value) { this.value = value; }
    T get() { return value; }
}

class Pair<K, V> {
    K key; V value;
    Pair(K key, V value) { this.key = key; this.value = value; }
}

static <T extends Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) >= 0 ? a : b;
}
```

18 Generics

~15 min

MINI PROBLEMS

1. Write a generic Pair<K, V> class and instantiate it with two different type combinations (e.g. Pair<String, Integer> and Pair<Integer, Integer>).

2. Write a generic max(T a, T b) method that works for any Comparable type, and test it with both Integer and String arguments.

Trainer Note: *Generics are a compile-time Java feature, not something a LeetCode judge exercises directly (its Java templates are already typed for you) — the two mini problems are the real practice here. §17's Design HashMap-style thinking (706, if you want a stretch problem) is a reasonable follow-on since real generic containers are exactly what such "Design" problems ask you to build.*

19 Streams

~15 min

CONCEPT

- The java.util.stream API lets you express operations over a collection declaratively — filter, map, reduce, collect — instead of writing manual for-loops with mutable accumulator variables.
- A stream pipeline is: source (a collection) -> zero or more intermediate operations (filter, map, sorted...) -> one terminal operation (collect, forEach, reduce...).

19 Streams

~15 min

REAL-LIFE EXAMPLE

ANALOGY

A factory assembly line: raw items move through a filtering station, then a transforming station, then a packaging station — each station does one job and passes the result on, rather than one worker handling every item from raw material to finished product by hand.

19 Streams

~15 min

JAVA SYNTAX

```
List<Integer> nums = List.of(1, 2, 3, 4, 5, 6);
List<Integer> evenSquares = nums.stream()
        .filter(n -> n % 2 == 0)
        .map(n -> n * n)
        .collect(Collectors.toList());
// evenSquares = [4, 16, 36]
```

19 Streams

~15 min

MINI PROBLEMS

1. Rewrite Day-1's fast-I/O sum/min/max logic using an IntStream over the array instead of a manual for-loop.

2. Given a List<String> of names, use a stream to produce a new list containing only names longer than 4 letters, each converted to uppercase.

Trainer Note: *LeetCode's Java judge doesn't require or reward Stream-based solutions, so there's no dedicated problem set here. As a style exercise, have students re-solve §17's 387 (First Unique Character in a String, Easy — https://leetcode.com/problems/first-unique-character-in-a-string/) using a Streams/Collectors frequency count instead of a manual loop, purely for practice with the API.*

20 Functional Java — Lambdas & Functional Interfaces

~15 min

CONCEPT

- A functional interface has exactly one abstract method (e.g. Runnable, Comparator<T>, Function<T,R>, Predicate<T>, Consumer<T>). A lambda expression is a short, inline implementation of that single method.
- Lambdas remove the boilerplate of writing a full named class (or anonymous inner class) just to pass around one small piece of behaviour.

20 Functional Java — Lambdas & Functional Interfaces

~15 min

REAL-LIFE EXAMPLE

ANALOGY

Handing a chef a small recipe card for one step ("reduce this sauce for 5 minutes") instead of hiring and training an entirely new chef every time you need a small task done — the lambda is that recipe card: minimal, disposable, and just enough instruction for the one job.

20 Functional Java — Lambdas & Functional Interfaces

~15 min

JAVA SYNTAX

```
// Custom functional interface
interface Calculator { int operate(int a, int b); }

Calculator add = (a, b) -> a + b;
Calculator multiply = (a, b) -> a * b;
System.out.println(add.operate(3, 4));       // 7

// Built-in functional interface: Comparator
List<Employee> staff = ...;
staff.sort((e1, e2) -> e1.name.compareTo(e2.name));
```

20 Functional Java — Lambdas & Functional Interfaces

~15 min

MINI PROBLEMS

1. Write the custom Calculator interface above and use lambdas to implement add, subtract, and multiply without writing three separate named classes.

2. Sort a List<Employee> two different ways (by name, then by salary) using two different lambda-based Comparators.

Trainer Note: *Lambdas are a Java syntax feature rather than something LeetCode grades directly — no dedicated problem set. As practice, have students re-solve any earlier sorting-based mini problem using a lambda Comparator in place of a named comparator class.*

21 Interface vs Abstract Class

~15 min

CONCEPT

- Interface: only method signatures (plus default/static methods with bodies since Java 8) and constants; no instance fields, no constructors; a class can implement many interfaces.
- Abstract class: can hold instance fields, constructors, and a mix of fully implemented and abstract methods; a class can extend only one abstract class.
- Rule of thumb: use an interface to describe a capability shared across otherwise-unrelated classes (Flyable, Comparable); use an abstract class to share common state/code among closely related classes (Shape holding a shared color field for Circle/Square).

21 Interface vs Abstract Class

~15 min

REAL-LIFE EXAMPLE

ANALOGY

Interface = a job description ("must be able to drive") that any qualified person can fulfil regardless of their background. Abstract class = a family recipe passed down through generations, where some steps are fixed by tradition and exactly one step is deliberately left for each descendant to customise.

21 Interface vs Abstract Class

~15 min

JAVA SYNTAX

```
interface Flyable { void fly(); }                  // pure contract

abstract class Shape {                              // shared state + partial implementation
    String color;
    Shape(String color) { this.color = color; }
    abstract double area();                         // must be customised
    void printColor() { System.out.println(color); }  // shared, ready-made
}
```

21 Interface vs Abstract Class

~15 min

MINI PROBLEMS

1. Given 3 short scenarios (e.g. "Bird and Airplane can both fly", "Circle and Square share a color field"), decide interface vs abstract class for each and justify the choice in one sentence.

2. Take an existing abstract class from earlier in the course and identify which parts would have to be dropped if you converted it into an interface instead.

21 Interface vs Abstract Class

~15 min

LEETCODE PRACTICE

622

Design Circular Queue Medium

Requires designing a clean operation contract first — good practice for deciding what belongs in an interface vs a full implementation.

707

Design Linked List Medium

A useful reuse from §12 here: notice how much of this class is shared state/behaviour (abstract-class style) vs a pure external contract (interface style).

22 Recursive Tower of Hanoi (+ Recursion Tree)

~20 min

CONCEPT

- Move n disks from a source rod to a destination rod, using an auxiliary rod, never placing a larger disk on a smaller one.
- Recursive idea: to move n disks from source to destination, first move the top n-1 disks from source to auxiliary, then move disk n directly from source to destination, then move the n-1 disks from auxiliary to destination. This is TWO recursive calls per level — a step up in complexity from Day-1's single-call recursion.
- Recurrence: $T(n) = 2 \cdot T(n-1) + 1$, giving $2^n - 1$ total moves.

22 Recursive Tower of Hanoi (+ Recursion Tree)

~20 min

REAL-LIFE EXAMPLE

ANALOGY

Moving a stack of nested gift boxes from one table to another, one box at a time, always keeping smaller boxes on top of larger ones, using a single small side table as temporary space partway through.

22 Recursive Tower of Hanoi (+ Recursion Tree)

~20 min

JAVA SYNTAX

```
void hanoi(int n, char from, char aux, char to) {
    if (n == 0) return;                 // base case
    hanoi(n - 1, from, to, aux);        // move top n-1 out of the way
    System.out.println("Move disk " + n + " from " + from + " to " + to);
    hanoi(n - 1, aux, from, to);        // move the n-1 disks onto destination
}
```

22 Recursive Tower of Hanoi (+ Recursion Tree)

~20 min

MINI PROBLEMS

1. By hand, draw the full recursion tree for hanoi(3, 'A', 'B', 'C') — every call and its two children.

2. Run hanoi(4, ...) and confirm the total move count equals $2^4 - 1 = 15$.

22 Recursive Tower of Hanoi (+ Recursion Tree)

~20 min

LEETCODE PRACTICE

Generate Parentheses Medium

22

There is no numbered LeetCode "Tower of Hanoi" problem (it lives on GeeksforGeeks/InterviewBit instead) — this is the closest available substitute: multi-branch recursion where each call spawns further recursive calls, the same structural leap Hanoi introduces today.

Trainer Note: *Being upfront: Tower of Hanoi itself is not an official numbered LeetCode problem — only discussion-forum posts reference it there. Assign it from this document / GfG for the graded hands-on, and use 22 (Generate Parentheses) purely as extra multi-branch-recursion practice.*

23 Recursive Sum of Digits (+ Recursion Tree)

~15 min

CONCEPT

- Single linear recursion: peel off the last digit, add it to the recursive result of the remaining (shorter) number, until the number reaches 0.
- This is the same shape as Day-1's live demo, but today it's a full graded hands-on task that must include a hand-drawn recursion tree.

23 Recursive Sum of Digits (+ Recursion Tree)

~15 min

REAL-LIFE EXAMPLE

ANALOGY

Counting a stack of coins one at a time: you count the top coin, then hand the rest of the stack to 'a smaller version of yourself' to count, trusting that smaller count comes back correct, then you add your one coin to it.

23 Recursive Sum of Digits (+ Recursion Tree)

~15 min

JAVA SYNTAX

```
int sumOfDigits(int n) {
    if (n == 0) return 0;                  // base case
    return (n % 10) + sumOfDigits(n / 10);
}
```

23 Recursive Sum of Digits (+ Recursion Tree)

~15 min

MINI PROBLEMS

1. Draw the full recursion tree (or in this case, recursion chain, since each call has only one child) for sumOfDigits(4321).

2. Modify the method to also return the number of recursive calls made, alongside the digit sum.

23 Recursive Sum of Digits (+ Recursion Tree)

~15 min

LEETCODE PRACTICE

Add Digits Easy

258

Direct extension of today's method — repeat the digit-sum process until a single digit remains.

24 Recursive String Reversal (+ Recursion Tree)

~15 min

CONCEPT

- Another single linear recursion: reverse the rest of the string first, then append the first character to the end (or equivalently, take the last character off and prepend it).
- Same base-case/shrink-the-problem discipline as sum-of-digits, applied to a String instead of an int.

24 Recursive String Reversal (+ Recursion Tree)

~15 min

REAL-LIFE EXAMPLE

ANALOGY

Unstacking a pile of papers from the top and placing each sheet at the back of a brand-new pile — repeating that single physical action once per sheet flips the entire order, exactly like one recursive call per character.

24 Recursive String Reversal (+ Recursion Tree)

~15 min

JAVA SYNTAX

```
String reverse(String s) {
    if (s.isEmpty()) return s;            // base case
    return reverse(s.substring(1)) + s.charAt(0);
}
```

24 Recursive String Reversal (+ Recursion Tree)

~15 min

MINI PROBLEMS

1. Draw the recursion tree/chain for reverse("CODE") showing what each call returns.

2. Rewrite the method using a StringBuilder accumulator parameter instead of string concatenation, and explain in one sentence why the accumulator version is more efficient (Strings are immutable — every + creates a new String).

24 Recursive String Reversal (+ Recursion Tree)

~15 min

LEETCODE PRACTICE

Reverse String Easy

344

Solve it with the recursive approach taught today (not the in-place two-pointer approach) to reinforce today's specific technique.

Reverse String II Easy

541

A variation that reverses only every alternate k-character block — good stretch problem once the basic recursive reversal is solid.

25 Constructor Overloading

~15 min

CONCEPT

- Multiple constructors in the same class with different parameter lists, letting an object be created with varying amounts of initial information.
- Overloaded constructors are commonly chained together with this() (§14) so the setup logic is written only once, in the most detailed constructor.

25 Constructor Overloading

~15 min

REAL-LIFE EXAMPLE

ANALOGY

Ordering a coffee: you can order with just a size, or size + flavour, or size + flavour + extra shots — each is a different 'constructor' for the same drink, offering a different amount of upfront detail.

25 Constructor Overloading

~15 min

JAVA SYNTAX

```
class Book {
    String title, author;
    double price;

    Book(String title) {
        this(title, "Unknown", 0.0);
    }
    Book(String title, String author) {
        this(title, author, 0.0);
    }
    Book(String title, String author, double price) {
        this.title = title; this.author = author; this.price = price;
    }
}
```

25 Constructor Overloading

~15 min

MINI PROBLEMS

1. Write the Book class above with all 3 overloaded constructors and instantiate one object from each constructor.

2. Write 3 overloaded constructors for a Rectangle class (no-arg, one-arg for a square, two-arg for length+width) and print the area from each.

Trainer Note: *Constructor design isn't something a single-function LeetCode submission format can assess — the two mini problems above, reviewed in notebooks, are the real assessment for this topic.*

26 Compile-Time vs Runtime Polymorphism (Practice)

~20 min

CONCEPT

- Reinforcing Day-1's definitions with harder tracing practice: compile-time polymorphism (overloading) is resolved by the compiler based on the declared argument types; runtime polymorphism (overriding) is resolved by the JVM based on the object's ACTUAL type, not the reference's declared type.
- Key trap to drill today: a variable declared as the parent type but pointing to a child object still runs the CHILD's overridden method when called — this is dynamic method dispatch.

26 Compile-Time vs Runtime Polymorphism (Practice)

~20 min

REAL-LIFE EXAMPLE

ANALOGY

A name tag that says 'Manager' but the actual person wearing it is a 'Regional Manager' — when you ask that person to do their job, what actually happens is the Regional Manager's version of the job, not a generic Manager's, even though the tag only says 'Manager'.

26 Compile-Time vs Runtime Polymorphism (Practice)

~20 min

JAVA SYNTAX

```
class Animal { void sound() { System.out.println("..."); } }
class Dog extends Animal {
    @Override void sound() { System.out.println("Bark"); }
}

Animal a = new Dog();    // reference type Animal, actual object Dog
a.sound();               // prints "Bark" — runtime polymorphism decides this, not the reference type
```

26 Compile-Time vs Runtime Polymorphism (Practice)

~20 min

MINI PROBLEMS

1. Given a short Shape -> Circle, Square hierarchy with both overloaded and overridden methods, predict the exact output of 5 given method calls BEFORE running the code, then verify.

2. Given 6 short code snippets, classify each as overloading, overriding, or neither, and justify each answer in one sentence.

26 Compile-Time vs Runtime Polymorphism (Practice)

~30 min

LEETCODE PRACTICE

Design a Stack With Increment Operation Medium

1381

Extends a familiar Stack contract (push/pop) with an added increment() behaviour — practice reasoning about "same interface, extended behaviour," the same mental muscle overriding uses.

Trainer Note: *LeetCode still can't directly grade "is this overloading or overriding" — the tracing/classification mini problems above are the real assessment; the LeetCode row is supporting practice for the surrounding design mindset.*

27 Method Overriding

~15 min

CONCEPT

- A subclass provides its own specific implementation of a method already defined in its superclass, with the identical signature. The @Override annotation is optional but strongly recommended — it makes the compiler check that you're actually overriding something (catches typos in the method signature).
- The most common real-world overrides are toString(), equals(), and hashCode() from the root Object class.

27 Method Overriding

~15 min

REAL-LIFE EXAMPLE

ANALOGY

A regional store franchise follows the parent company's standard checkout process template exactly, except it overrides just the 'apply local tax' step to match local regulations — everything else about the checkout stays the same as the standard.

27 Method Overriding

~15 min

JAVA SYNTAX

```
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public String toString() { return "Point(" + x + ", " + y + ")"; }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Point p)) return false;
        return x == p.x && y == p.y;
    }
    @Override
    public int hashCode() { return Objects.hash(x, y); }
}
```

27 Method Overriding

~15 min

MINI PROBLEMS

1. Override toString(), equals(), and hashCode() in a Point class, then verify that a HashSet<Point> correctly treats two Points with the same x, y as duplicates.

2. Create two child classes that each override a parent's calculateBonus() method differently, then loop over a List<Employee> (parent-typed references) and print each object's polymorphic bonus.

Trainer Note: *No LeetCode problem number specifically grades "did you correctly override a method" — the two mini problems above (especially the equals()/hashCode() contract check via HashSet) are the real-world-relevant assessment for this topic.*

27 CONCEPTS COVERED

End of Day 1

Every mini problem and LeetCode set in this deck maps back to the Day-1 Concept Reference (Combined) document — use that for full trainer notes, analogies, and pacing guidance.